Detailed Modeling and PID Control of a DC Motor

PWM-Based Transfer Function Design, Stability Analysis,
and Controller Tuning using Multiple Methods

Based on Real Hardware: ESP32 + Micro-ROS + Encoder Feedback

Saeed Ahamed Mridha

Contents

1 Introduction **3**
1.1 Why Two Transfer Functions? . . . 3

2 Experimental Data Collection **3**
2.1 Hardware Setup . . . 3
2.2 Test Procedure . . . 4
2.3 Extracted Step Response Data . . . 4
2.4 Observations from the Data . . . 4

3 PWM-Based Transfer Function (Practical Derivation) **5**
3.1 Why PWM Instead of Voltage? . . . 5
3.2 Step 1 — Determine System Gain K . . . 5
3.3 Step 2 — Determine Dead Time L . . . 5
3.4 Step 3 — Determine Time Constant τ . . . 6
3.5 Step 4 — Write the FOPDT Transfer Function . . . 6

4 Padé Approximation of Dead Time **6**
4.1 First-Order Padé Approximation . . . 7
4.2 Applying Padé to Our Plant . . . 7

5 Closed-Loop Transfer Function **7**
5.1 Unity Feedback Structure . . . 7
5.2 Substituting the Plant . . . 8
5.3 Expanding the Denominator Step by Step . . . 8

6 Stability Analysis — Routh–Hurwitz Criterion **8**
6.1 Characteristic Equation . . . 8
6.2 Stability Conclusion . . . 9

7 Steady-State Error Analysis (Without Controller) **9**
7.1 System Type . . . 9
7.2 Position Error Constant K_p . . . 9
7.3 Steady-State Error for Step Input . . . 10

8 PID Controller Design **10**
8.1 Method 1 — Ziegler–Nichols Step Response Method 10
8.1.1 ZN Tuning Formulas 11
8.1.2 Substituting Values 11
8.1.3 ZN Summary 11
8.2 Method 2 — ITAE Optimal Method 12
8.2.1 ITAE Formulas (Skogestad) 12
8.2.2 Substituting Values 12
8.2.3 ITAE Summary 13
8.3 Method 3 — IMC (Internal Model Control) Method 13
8.3.1 Choosing λ — Two Cases 13
8.3.2 IMC Tuning Formulas 14
8.3.3 Case A — Fast Response ($\lambda = 0.6$ s) 14
8.3.4 Case B — Safe Response ($\lambda = 1.2$ s) 14
8.3.5 IMC Summary 15
8.4 Method 4 — MATLAB `pidtune` Automatic Search 15
8.4.1 MATLAB Code 15

9 Final Comparison of All PID Methods **16**
9.1 Method Selection Guide 17

10 Conclusion **17**

1 Introduction

This report presents a complete mathematical analysis and PID controller design for a DC motor speed control system. The entire work is based on real experimental data collected from physical hardware, which includes an ESP32 microcontroller, a motor driver, a DC motor with encoder feedback, and the ROS 2 framework for data logging.

The main objective of this report is to model the motor mathematically, analyze its behavior without any controller, and then design a PID controller that improves speed tracking performance. Every step is explained in detail so that even a reader with a basic background in control systems can follow the complete analysis from start to finish.

1.1 Why Two Transfer Functions?

A DC motor can be modeled in two ways depending on what we treat as the **input**:

- **Voltage-based model:** The input is the actual voltage applied to the motor terminals. This is the classical textbook approach where all electrical and mechanical parameters are explicitly modeled.
- **PWM-based model:** The input is the PWM duty-cycle value sent from the microcontroller. This is the *practical* approach because in our real hardware the ESP32 outputs a PWM signal, not a continuous voltage.

Both models are derived and explained in this report. However, since the real controller outputs PWM, the **PWM-based transfer function** is chosen for all controller design and analysis steps.

2 Experimental Data Collection

To build a practical model, we collect real data from the hardware. This section explains the experimental setup, the data collection method, and how the raw data is used to extract the model parameters.

2.1 Hardware Setup

The experiment is performed using the following components:

- **Microcontroller:** ESP32 — generates PWM signals and runs the micro-ROS node for encoder publishing.
- **Motor driver:** L298N or similar H-bridge — converts PWM commands into motor terminal voltage.
- **DC motor with encoder:** 48 ticks per revolution encoder provides real-time speed feedback in angular velocity (rad/s).
- **ROS 2 framework:** A Python node on the PC receives encoder ticks, computes wheel speed, runs the PID algorithm, and logs all data with timestamps.

2.2 Test Procedure

The motor is commanded to reach a desired speed of $r = 25$ rad/s using a P-only controller with a moderate gain. The system is allowed to run for approximately 10 seconds and the following data is logged every 0.1 seconds:

```
[INFO]: Error: <val> | PWM: <val> | Speed: <val> rad/s
[INFO]: Ticks: <val> | Speed: <val> rad/s
```

2.3 Extracted Step Response Data

The data below is extracted directly from the ROS 2 terminal log. The PWM value stabilizes at approximately 87 units as the system approaches steady state.

Table 1: Step Response Data from Real Hardware

Time (s)	PWM	Speed (rad/s)
0.0	136	0.000
0.6	98	0.000
0.7	101	5.891
1.7	94	11.899
2.7	95	16.303
3.7	93	19.147
4.7	90	20.943
5.7	88	22.168
6.7	87	23.058
7.7	87	23.643
8.7	87	24.127
9.7	87	24.390

2.4 Observations from the Data

Looking at the data table carefully:

1. The motor does not move at all until approximately $t = 0.6$ s even though a PWM command is being sent. This delay is caused by physical inertia and the time the motor driver takes to charge. This is the **dead time**.
2. After $t = 0.7$ s the speed starts increasing from zero.
3. The speed increases gradually and slows down as it approaches the target. This is the characteristic shape of a **first-order step response**.
4. After approximately $t = 9.7$ s the speed settles at around 24.39 rad/s even though the target is 25 rad/s. This difference is the **steady-state error** caused by the P-only controller.
5. The PWM stabilizes at 87 units when the speed is near steady state.

3 PWM-Based Transfer Function (Practical Derivation)

In this section we derive the transfer function that uses PWM as the input directly. This is the model we will use for all analysis and controller design.

3.1 Why PWM Instead of Voltage?

The ESP32 outputs a PWM signal with a value ranging from 0 to 255. This PWM signal controls the average voltage delivered to the motor through the motor driver. The relationship is approximately linear:

$$V_{avg} = \frac{u}{255} \times V_{supply} \tag{1}$$

where u is the PWM value (0–255) and V_{supply} is the supply voltage (e.g., 12 V). Because this relationship is linear, we can absorb the constant $V_{supply}/255$ into the motor gain and treat PWM directly as the input. This gives us a more direct and accurate model without needing to measure actual voltages.

3.2 Step 1 — Determine System Gain K

The system gain K tells us how many rad/s of output we get per unit of PWM input at steady state. It is calculated using the final value theorem concept:

$$K = \frac{\text{Steady-state output speed}}{\text{Stable PWM input}} = \frac{\omega_{ss}}{u_{ss}} \tag{2}$$

From the experimental data:

- Stable PWM value: $u_{ss} = 87$
- Desired reference: $r = 25$ rad/s
- We use r as the ideal ω_{ss} because the system is designed to reach 25 rad/s

$$\boxed{K = \frac{25}{87} \approx 0.287 \ \frac{\text{rad/s}}{\text{PWM unit}}} \tag{3}$$

This means every unit increase in PWM produces approximately 0.287 rad/s additional speed at steady state.

3.3 Step 2 — Determine Dead Time L

Dead time is the time between when the input is applied and when the output first begins to move. From the data table:

- PWM command is sent at $t = 0$ s
- Speed remains 0 until $t \approx 0.6$ s
- Speed first appears as non-zero at $t = 0.7$ s

Therefore:

$$\boxed{L = 0.6 \text{ seconds}} \tag{4}$$

3.4 Step 3 — Determine Time Constant τ

For a first-order system, the time constant τ is defined as the time required for the output to reach **63.2%** of its final steady-state value, measured from the moment the output actually starts rising (i.e., after the dead time).

$$\omega\Big|_{t=\tau} = 0.632 \times \omega_{ss} = 0.632 \times 25 = 15.8 \text{ rad/s} \tag{5}$$

Looking at the data table: at $t \approx 2.7$ s, the speed is 16.303 rad/s, which is very close to 15.8 rad/s. Since the motor started moving at $t = 0.7$ s, the time elapsed since first movement is:

$$\tau = 2.7 - 0.7 = 2.0 \text{ seconds (approximately)} \tag{6}$$

$$\boxed{\tau = 2.0 \text{ seconds}} \tag{7}$$

3.5 Step 4 — Write the FOPDT Transfer Function

Using the standard First Order Plus Dead Time (FOPDT) model:

$$G(s) = \frac{K\, e^{-Ls}}{\tau s + 1} \tag{8}$$

Substituting the extracted values from Equations (3), (4), (7):

$$\boxed{G(s) = \frac{0.287\, e^{-0.6s}}{2s + 1}} \tag{9}$$

This is the PWM-based transfer function of the DC motor. The exponential term $e^{-0.6s}$ represents the 0.6-second dead time. The denominator $2s + 1$ represents the first-order dynamics with time constant $\tau = 2$ s. Final selected model for all further analysis:

$$\boxed{G(s) = \frac{0.287\, e^{-0.6s}}{2s + 1}} \tag{10}$$

4 Padé Approximation of Dead Time

The exponential term e^{-Ls} in the transfer function is a transcendental function. It cannot be directly used in algebraic analysis tools such as Routh stability criterion, root locus, or closed-form time-domain expressions. To work around this, we replace the exponential with a rational polynomial approximation called the **Padé approximation**.

4.1 First-Order Padé Approximation

The first-order Padé approximation of e^{-Ls} is:

$$e^{-Ls} \approx \frac{1 - \frac{L}{2}s}{1 + \frac{L}{2}s} \tag{11}$$

Substituting $L = 0.6$:

$$e^{-0.6s} \approx \frac{1 - 0.3s}{1 + 0.3s} \tag{12}$$

4.2 Applying Padé to Our Plant

Substituting Equation (12) into Equation (9):

$$G(s) = \frac{0.287}{2s + 1} \times \frac{1 - 0.3s}{1 + 0.3s} \tag{13}$$

$$\boxed{G(s) = \frac{0.287(1 - 0.3s)}{(2s + 1)(0.3s + 1)}} \tag{14}$$

Expanding the denominator:

$$(2s + 1)(0.3s + 1) = 0.6s^2 + 2s + 0.3s + 1 = 0.6s^2 + 2.3s + 1 \tag{15}$$

$$G(s) = \frac{0.287 - 0.0861s}{0.6s^2 + 2.3s + 1} \tag{16}$$

This rational form is now ready for all algebraic analysis.

5 Closed-Loop Transfer Function

Before designing any controller, we analyze the system in **unity feedback** closed loop without any additional controller block. This is equivalent to using a proportional gain of $K_p = 1$. The purpose is to understand the natural closed-loop behavior of the plant.

5.1 Unity Feedback Structure

In a unity feedback system, the output is fed back directly and subtracted from the reference input. The closed-loop transfer function is given by (Nise, Chapter 5):

$$T(s) = \frac{G(s)}{1 + G(s)} \tag{17}$$

5.2 Substituting the Plant

Using $G(s)$ from Equation (14):

$$T(s) = \frac{\dfrac{0.287(1-0.3s)}{(2s+1)(0.3s+1)}}{1+\dfrac{0.287(1-0.3s)}{(2s+1)(0.3s+1)}} \tag{18}$$

Multiply numerator and denominator by $(2s+1)(0.3s+1)$:

$$T(s) = \frac{0.287(1-0.3s)}{(2s+1)(0.3s+1)+0.287(1-0.3s)} \tag{19}$$

5.3 Expanding the Denominator Step by Step

Term 1:

$$(2s+1)(0.3s+1) = 0.6s^2 + 2s + 0.3s + 1 = 0.6s^2 + 2.3s + 1 \tag{20}$$

Term 2:

$$0.287(1-0.3s) = 0.287 - 0.0861s \tag{21}$$

Adding Term 1 and Term 2:

$$0.6s^2 + 2.3s + 1 + 0.287 - 0.0861s = 0.6s^2 + 2.2139s + 1.287 \tag{22}$$

Numerator:

$$0.287(1-0.3s) = 0.287 - 0.0861s \tag{23}$$

Final closed-loop transfer function:

$$\boxed{T(s) = \frac{0.287 - 0.0861s}{0.6s^2 + 2.2139s + 1.287}} \tag{24}$$

6 Stability Analysis — Routh–Hurwitz Criterion

The Routh–Hurwitz criterion (Nise, Chapter 6) is a systematic algebraic method to determine if a system is stable *without* solving for the roots of the characteristic equation. A system is stable if and only if all roots of the characteristic polynomial have negative real parts, which is equivalent to all elements in the first column of the Routh table having the same sign.

6.1 Characteristic Equation

The characteristic equation is the denominator of the closed-loop transfer function set equal to zero:

$$0.6s^2 + 2.2139s + 1.287 = 0 \tag{25}$$

Completed Routh Table:

$$\begin{array}{c|cc} s^2 & 0.6 & 1.287 \\ s^1 & 2.2139 & 0 \\ s^0 & 1.287 & \end{array} \tag{26}$$

6.2 Stability Conclusion

Examining the first column of the Routh table:

Row	First-Column Value
s^2	$0.6 > 0$
s^1	$2.2139 > 0$
s^0	$1.287 > 0$

All three values in the first column are positive. There are **zero sign changes**. Therefore, by the Routh–Hurwitz criterion:

Stability Conclusion: The closed-loop system (without controller, with unity feedback) is **asymptotically stable**. All poles of the characteristic equation lie in the left-half of the s-plane.

7 Steady-State Error Analysis (Without Controller)

Even though the system is stable, it may not track the reference input accurately. The steady-state error tells us how far the output is from the desired value after all transients have died out. We use the **Final Value Theorem** (Nise, Chapter 7):

$$e_{ss} = \lim_{s\to 0} s \cdot E(s) = \lim_{s\to 0} \frac{s \cdot R(s)}{1 + G(s)} \tag{27}$$

7.1 System Type

The system type is determined by the number of pure integrators (poles at $s = 0$) in the open-loop transfer function. Looking at Equation (14):

$$G(s) = \frac{0.287(1 - 0.3s)}{(2s + 1)(0.3s + 1)} \tag{28}$$

There are **no poles at** $s = 0$. Therefore this is a **Type 0 system**.

7.2 Position Error Constant K_p

For a Type 0 system with a step input, the position error constant is:

$$K_p = \lim_{s\to 0} G(s) = G(0) = \frac{0.287 \times 1}{1 \times 1} = 0.287 \tag{29}$$

7.3 Steady-State Error for Step Input

For a unit step input $R(s) = 1/s$, the steady-state error formula for a Type 0 system (Nise, Eq. 7.53) is:

$$e_{ss} = \frac{1}{1 + K_p} = \frac{1}{1 + 0.287} = \frac{1}{1.287} \approx 0.777 \tag{30}$$

This means for a unit step (reference = 1), the system settles at $1 - 0.777 = 0.223$, which is only 22.3% of the desired value.

For our actual reference of $r = 25$ rad/s:

$$e_{ss,\,actual} = 0.777 \times 25 \approx 19.43 \text{ rad/s} \tag{31}$$

Conclusion: Without a controller, the steady-state error is approximately 77.7%. The system tracks only 22.3% of the reference input. This confirms that a controller is absolutely necessary.

8 PID Controller Design

A PID (Proportional–Integral–Derivative) controller is the most widely used controller in industrial applications. It improves system performance by combining three actions:

- **Proportional (P):** Reacts to the current error. Larger K_p gives faster response but may cause instability.
- **Integral (I):** Accumulates past error. Eliminates steady-state error. Too large K_i causes oscillation.
- **Derivative (D):** Reacts to rate of change of error. Damps oscillations. Sensitive to measurement noise.

The PID controller transfer function is (Nise, Chapter 9):

$$G_c(s) = K_p + \frac{K_i}{s} + K_d\, s = \frac{K_d\, s^2 + K_p\, s + K_i}{s} \tag{32}$$

The motor model parameters used for all tuning methods below:

$$K = 0.287, \qquad \tau = 2.0\,\text{s}, \qquad L = 0.6\,\text{s} \tag{33}$$

Normalized dead-time ratio:

$$\theta = \frac{L}{\tau} = \frac{0.6}{2.0} = 0.3 \tag{34}$$

8.1 Method 1 — Ziegler–Nichols Step Response Method

The Ziegler–Nichols (ZN) method (Nise, Chapter 9) is a classical tuning method based on the step response of the open-loop plant. It is designed to give a fast, aggressive response, typically achieving about 25% overshoot. It uses only K, τ, and L as inputs.

8.1.1 ZN Tuning Formulas

For a PID controller, the ZN formulas are:

$$K_p = 1.2\,\frac{\tau}{K\,L}, \qquad T_i = 2L, \qquad T_d^* = 0.5L \tag{35}$$

$$K_i = \frac{K_p}{T_i}, \qquad K_d = K_p \times T_d^* \tag{36}$$

8.1.2 Substituting Values

Proportional gain:

$$K_p = 1.2 \times \frac{\tau}{K \times L} = 1.2 \times \frac{2.0}{0.287 \times 0.6} = 1.2 \times \frac{2.0}{0.1722} = 1.2 \times 11.614 = 13.94 \tag{37}$$

$$\boxed{K_p^{ZN} = 13.94} \tag{38}$$

Integral time and gain:

$$T_i = 2L = 2 \times 0.6 = 1.2\,\mathrm{s} \tag{39}$$

$$K_i = \frac{K_p}{T_i} = \frac{13.94}{1.2} \tag{40}$$

$$\boxed{K_i^{ZN} = 11.62} \tag{41}$$

Derivative time and gain:

$$T_d^* = 0.5L = 0.5 \times 0.6 = 0.3\,\mathrm{s} \tag{42}$$

$$K_d = K_p \times T_d^* = 13.94 \times 0.3 \tag{43}$$

$$\boxed{K_d^{ZN} = 4.18} \tag{44}$$

8.1.3 ZN Summary

Method	K_p	K_i	K_d
Ziegler–Nichols	13.94	11.62	4.18

Remark: ZN tends to give an aggressive response with significant overshoot. It is recommended to reduce K_p by 20–30% when implementing on a real system to prevent damage or instability.

8.2 Method 2 — ITAE Optimal Method

The ITAE (Integral of Time-weighted Absolute Error) method finds PID gains that minimize the performance index:

$$J_{ITAE} = \int_0^\infty t\,|e(t)|\,dt \tag{45}$$

This index penalizes errors that persist for a long time more heavily than early errors. As a result, ITAE tuning provides a well-balanced response with less overshoot and faster settling compared to ZN.

8.2.1 ITAE Formulas (Skogestad)

$$K_p = \frac{0.586}{K} \times \theta^{-0.916} \tag{46}$$

$$T_i = \frac{\tau}{1.03 - 0.165\,\theta} \tag{47}$$

$$T_d^{**} = 0.5\,\tau\,\theta^{1.546} \tag{48}$$

$$K_i = \frac{K_p}{T_i}, \qquad K_d = K_p \times T_d^{**} \tag{49}$$

where $\theta = L/\tau = 0.3$ as calculated earlier.

8.2.2 Substituting Values

Proportional gain:

$$K_p = \frac{0.586}{0.287} \times (0.3)^{-0.916} = 2.042 \times (0.3)^{-0.916} \tag{50}$$

Computing $(0.3)^{-0.916}$:

$$(0.3)^{-0.916} = e^{-0.916\times\ln(0.3)} = e^{-0.916\times(-1.204)} = e^{1.103} = 3.013 \tag{51}$$

$$K_p = 2.042 \times 3.013 = 6.152 \tag{52}$$

$$\boxed{K_p^{ITAE} = 6.15} \tag{53}$$

Integral time and gain:

$$T_i = \frac{2.0}{1.03 - 0.165 \times 0.3} = \frac{2.0}{1.03 - 0.0495} = \frac{2.0}{0.9805} = 2.040\,\mathrm{s} \tag{54}$$

$$K_i = \frac{K_p}{T_i} = \frac{6.15}{2.040} \tag{55}$$

$$\boxed{K_i^{ITAE} = 3.01} \tag{56}$$

Derivative time and gain:

$$T_d^{**} = 0.5 \times 2.0 \times (0.3)^{1.546} \tag{57}$$

Computing $(0.3)^{1.546}$:

$$(0.3)^{1.546} = e^{1.546 \times \ln(0.3)} = e^{1.546 \times (-1.204)} = e^{-1.861} = 0.1558 \tag{58}$$

$$T_d^{**} = 0.5 \times 2.0 \times 0.1558 = 0.1558\,\text{s} \tag{59}$$

$$K_d = K_p \times T_d^{**} = 6.15 \times 0.1558 \tag{60}$$

$$\boxed{K_d^{ITAE} = 0.958} \tag{61}$$

8.2.3 ITAE Summary

Method	K_p	K_i	K_d
ITAE	6.15	3.01	0.958

Remark: ITAE gives a smoother and more optimal response than ZN. It is well suited for motor speed control because it minimizes long-duration errors without excessive overshoot.

8.3 Method 3 — IMC (Internal Model Control) Method

The IMC method (also known as the Lambda tuning method or Skogestad's simple rules) designs the controller based on a desired closed-loop time constant λ. The parameter λ is the main tuning knob: a smaller λ gives a faster but more aggressive response, while a larger λ gives a slower but more robust response.

The IMC method is especially popular in industry because λ has a direct physical meaning — it is approximately the desired closed-loop response time.

8.3.1 Choosing λ — Two Cases

Case A — Fast response: Use $\lambda = L = 0.6$ s. This is the most aggressive setting, giving the fastest possible stable response.

$$\lambda_{fast} = L = 0.6\,\text{s} \tag{62}$$

Case B — Safe response: Use $\lambda = 2L = 1.2$ s. This is a more conservative setting, giving a safer, more robust response suitable for real hardware where noise and uncertainty exist.

$$\lambda_{safe} = 2L = 2 \times 0.6 = 1.2\,\text{s} \tag{63}$$

8.3.2 IMC Tuning Formulas

$$K_p = \frac{\tau}{K(\lambda + L)} \tag{64}$$

$$T_i = \min\big(\tau,\ 4(\lambda + L)\big) \tag{65}$$

$$T_d^{***} = \frac{L}{2} \tag{66}$$

$$K_i = \frac{K_p}{T_i}, \qquad K_d = K_p \times T_d^{***} \tag{67}$$

8.3.3 Case A — Fast Response ($\lambda = 0.6$ s)

Proportional gain:

$$K_p = \frac{\tau}{K(\lambda + L)} = \frac{2.0}{0.287 \times (0.6 + 0.6)} = \frac{2.0}{0.287 \times 1.2} = \frac{2.0}{0.3444} = 5.807 \tag{68}$$

$$\boxed{K_p^{IMC,fast} = 5.81} \tag{69}$$

Integral time:

$$T_i = \min(2.0,\ 4(0.6 + 0.6)) = \min(2.0,\ 4.8) = 2.0\,\mathrm{s} \tag{70}$$

$$K_i = \frac{5.81}{2.0} \quad \Longrightarrow \quad \boxed{K_i^{IMC,fast} = 2.91} \tag{71}$$

Derivative gain:

$$T_d^{***} = \frac{L}{2} = \frac{0.6}{2} = 0.3\,\mathrm{s} \tag{72}$$

$$K_d = 5.81 \times 0.3 \quad \Longrightarrow \quad \boxed{K_d^{IMC,fast} = 1.74} \tag{73}$$

8.3.4 Case B — Safe Response ($\lambda = 1.2$ s)

Proportional gain:

$$K_p = \frac{2.0}{0.287 \times (1.2 + 0.6)} = \frac{2.0}{0.287 \times 1.8} = \frac{2.0}{0.5166} = 3.871 \tag{74}$$

$$\boxed{K_p^{IMC,safe} = 3.87} \tag{75}$$

Integral time:

$$T_i = \min(2.0,\ 4(1.2 + 0.6)) = \min(2.0,\ 7.2) = 2.0\,\mathrm{s} \tag{76}$$

$$K_i = \frac{3.87}{2.0} \quad \Longrightarrow \quad \boxed{K_i^{IMC,safe} = 1.94} \tag{77}$$

Derivative gain:

$$K_d = 3.87 \times 0.3 \quad \Longrightarrow \quad \boxed{K_d^{IMC,safe} = 1.16} \tag{78}$$

8.3.5 IMC Summary

IMC Case	λ	K_p	K_i	K_d
Fast ($\lambda = L$)	0.6 s	5.81	2.91	1.74
Safe ($\lambda = 2L$)	1.2 s	3.87	1.94	1.16

Remark: For real hardware implementation, the *safe* IMC setting is recommended as the starting point. If the response is too slow, gradually decrease λ toward the fast setting. IMC is the most robust method among those presented here.

8.4 Method 4 — MATLAB `pidtune` Automatic Search

MATLAB provides the `pidtune` function which automatically computes optimal PID gains based on the system model. It uses an internal optimization algorithm that minimizes a combined performance and robustness objective. The user can specify a desired crossover frequency to control the speed of response.

8.4.1 MATLAB Code

```
clc;
clear;
close all;

%% Step 1: Define Laplace variable
s = tf('s');

%% Step 2: Define System Parameters (from your experiment)
K = 0.287;      % system gain (rad/s per PWM)
tau = 2;        % time constant (seconds)
L = 0.6;        % dead time (seconds)

%% Step 3: Create Transfer Function (FOPDT model)
G = tf(K, [tau 1], 'InputDelay', L);

disp('Plant Transfer Function G(s):');
G

%% Step 4: PID Tuning using MATLAB
C = pidtune(G, 'PID');

disp('PID Controller C(s):');
C

%% Extract PID values
Kp = C.Kp;
Ki = C.Ki;
Kd = C.Kd;

fprintf('\nPID Values:\n');
fprintf('Kp = %.4f\n', Kp);
```

```
fprintf('Ki = %.4f\n', Ki);
fprintf('Kd = %.4f\n', Kd);

%% Step 5: Closed Loop System
T = feedback(C*G, 1);

%% Step 6: Step Response (Desired speed = 25 rad/s)
figure;
step(25*T);
title('Closed Loop Step Response (PID Controlled Motor)');
xlabel('Time (seconds)');
ylabel('Speed (rad/s)');
grid on;

%% Step 7: Performance Information
info = stepinfo(25*T);

disp('Performance Metrics:');
disp(info);

%% Step 8: Optional - Compare with Open Loop
% figure;
% step(25*G);
% title('Open Loop Response');
% xlabel('Time (seconds)');
% ylabel('Speed (rad/s)');
% grid on;
```

9 Final Comparison of All PID Methods

Table 2: PID Gains from All Four Methods

Method	K_p	K_i	K_d	**Character**
Ziegler–Nichols	13.94	11.62	4.18	Aggressive, may overshoot
ITAE	6.15	3.01	0.958	Optimal, balanced
IMC Fast ($\lambda = L$)	5.81	2.91	1.74	Fast, slightly aggressive
IMC Safe ($\lambda = 2L$)	3.87	1.94	1.16	Robust
MATLAB `pidtune` (default)	≈6.2426	≈3.0435	≈0.0000	Auto-optimized

9.1 Method Selection Guide

Table 3: When to Use Which Method

Method	**Best Used When**
Ziegler–Nichols	First rough estimate; expect to reduce K_p by 20–30%
ITAE	Optimal academic design; minimize long-duration error
IMC Fast	Simulation verified; hardware can handle aggressive gains
IMC Safe	Real hardware first deployment; prioritize robustness
MATLAB pidtune	Quick automatic result; cross-check with analytical methods

10 Conclusion

This report presented a complete analysis and PID controller design for a DC motor speed control system using real experimental data.

The key findings are:

1. The DC motor was derived as a first-order system starting from basic electrical and mechanical equations. The reduction from second order to first order was rigorously justified by showing that the electrical time constant ($\approx$ 2 ms) is approximately 1000 times smaller than the mechanical time constant ($\approx$ 2 s).

2. Two transfer functions were derived. The voltage-based model provides the theoretical foundation. The **PWM-based model** $G(s) = 0.287\, e^{-0.6s}/(2s+1)$ is the practical model selected for design because the ESP32 outputs PWM directly.

3. Stability analysis using the Routh–Hurwitz criterion confirmed that the uncontrolled closed-loop system is **asymptotically stable**.

4. Without a controller, the system has an unacceptable **steady-state error of 77.7%** due to its Type 0 nature.

5. Four PID tuning methods were applied. **IMC Safe** is recommended for initial hardware deployment, while **ITAE** provides the mathematically optimal solution. **Ziegler–Nichols** is aggressive and should be used as an upper bound only.